

HOLLISTER INCORPORATED

PLC-to-MES Integration Technical Specification

*Vendor-Neutral Abstraction Architecture
for IIoT, Edge Computing, and Digital Twin Enablement*

Document ID	HOLL-TECH-PLC-MES-001
Version	1.0
Date	February 2026
Classification	Confidential
Status	Draft for Review

Companion Document to:
Digital Factory Solution Architecture (HOLL-ARCH-001)

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Design Principles	4
2. Architecture Overview	5
2.1 Layered Architecture Model	5
2.2 Data Flow Architecture	5
3. Physical Device Layer (Layer 1)	6
3.1 PLC Configuration Standards	6
3.1.1 Supported PLC Platforms	6
3.1.2 Tag Naming Convention	6
3.1.3 Tag Naming Elements	6
3.1.4 Equipment State Model (PackML/ISA-88)	7
4. Edge Computing Layer (Layer 3)	8
4.1 Edge Gateway Architecture	8
4.1.1 Edge Gateway Functions	8
4.1.2 Recommended Edge Platforms	8
4.2 Protocol Standards	8
4.2.1 OPC-UA (Primary Standard)	8
4.2.2 MQTT (Event Messaging)	9
5. Semantic and Context Layer (Layer 4)	10
5.1 Canonical Data Model	10
5.1.1 Equipment Hierarchy (ISA-95)	10
5.1.2 Equipment Definition Schema	10
5.2 Metadata Registry	12

5.2.1 Registry Components	12
5.3 Data Governance	12
5.3.1 Governance Policies	12
6. Integration Platform (Layer 5)	13
6.1 Event Streaming Architecture	13
6.1.1 Kafka Topic Structure	13
6.1.2 Message Schema (Avro)	13
6.2 API Gateway	15
6.2.1 API Endpoints	15
7. MES Integration (Layer 6)	16
7.1 MES Adapter Pattern	16
7.1.1 Adapter Responsibilities	16
7.1.2 Critical Manufacturing MES Integration Points	16
7.1.3 Adapter Implementation Pattern	16
8. Security Architecture	19
8.1 OT/IT Network Segmentation	19
8.1.1 Network Zones	19
8.2 Authentication and Authorization	19
8.3 Regulatory Compliance (21 CFR Part 11)	19
9. Implementation Guide	20
9.1 Equipment Onboarding Checklist	20
9.2 Troubleshooting Guide	20
Appendix A: OPC-UA Information Models	21
Appendix B: Reference Architecture Diagram	22

1. Introduction

1.1 Purpose

This Technical Specification defines the architecture, standards, and implementation guidelines for integrating Programmable Logic Controllers (PLCs) and manufacturing equipment with the Manufacturing Execution System (MES) across all Hollister manufacturing facilities. The specification establishes a vendor-neutral abstraction layer that enables interoperability, prevents vendor lock-in, and provides the foundation for Industrial Internet of Things (IIoT), edge computing, and Digital Twin capabilities.

1.2 Scope

This specification covers:

- PLC configuration standards and tag naming conventions
- Edge computing architecture and protocol translation
- Semantic modeling and canonical data structures
- Event-driven integration platform design
- MES integration patterns and adapters
- Data governance and regulatory compliance
- Security architecture for OT/IT convergence

1.3 Design Principles

The architecture is built on the following foundational principles:

Principle	Description
Abstraction	No direct coupling between MES and physical devices. All integration flows through abstraction layers that normalize protocols and data models.
Standards-Based	Use industry standards (OPC-UA, MQTT, ISA-95, ISA-88) over proprietary protocols wherever possible.
Event-Driven	Publish-subscribe architecture decouples data producers from consumers, enabling future extensibility.
Semantic Context	All data carries meaning through canonical models, metadata, and governed vocabularies.
Edge Intelligence	Process data close to the source for latency, bandwidth, and resilience benefits.
Governance First	Audit trails, data lineage, and compliance controls are built in from day one.

2. Architecture Overview

2.1 Layered Architecture Model

The PLC-to-MES integration architecture consists of six distinct layers, each with specific responsibilities. Data flows upward from physical devices through abstraction and context layers to consuming applications.

Layer	Name	Responsibility
6	Application Layer	MES, Digital Twin, Analytics, AI/ML consumers
5	Integration Platform	Event streaming (Kafka), API gateway, message routing
4	Semantic/Context Layer	Canonical models, metadata registry, governance engine
3	Edge Computing Layer	Protocol translation, data normalization, local processing
2	Communication Layer	OPC-UA servers, MQTT brokers, industrial protocols
1	Physical Device Layer	PLCs, sensors, actuators, equipment controllers

2.2 Data Flow Architecture

Data flows through the architecture in both directions:

Northbound (Device to Enterprise):

- Equipment state changes, production counts, process measurements
- Quality inspection results, alarm events
- Material consumption and tracking events

Southbound (Enterprise to Device):

- Work order downloads and recipe parameters
- Setpoint changes and production commands
- Label data and serialization assignments

3. Physical Device Layer (Layer 1)

3.1 PLC Configuration Standards

All PLCs integrated into the Digital Factory architecture must conform to these configuration standards to ensure consistent integration and future portability.

3.1.1 Supported PLC Platforms

Vendor	Platform	Protocol	Status
Rockwell/Allen-Bradley	ControlLogix, CompactLogix	EtherNet/IP, CIP	Preferred
Siemens	S7-1500, S7-1200	PROFINET, S7 Protocol	Supported
Schneider Electric	Modicon M580, M340	Modbus TCP, EtherNet/IP	Supported
Beckhoff	TwinCAT 3	ADS, OPC-UA native	Supported
Omron	NX/NJ Series	EtherNet/IP, EtherCAT	Supported

3.1.2 Tag Naming Convention

A standardized tag naming convention is critical for abstraction. All PLC tags must follow this hierarchical naming pattern:

```
[Site]_[Area]_[Equipment]_[Module]_[Point]_[Attribute]
```

Examples:

BAL_MH1_MOLD01_HYD_PRESSURE_PV	(Ballina, Molding Hall 1, Mold 01, Hydraulic, Pressure, Process Value)
BAL_MH1_MOLD01_HYD_PRESSURE_SP	(Setpoint)
BAL_MH1_MOLD01_HYD_PRESSURE_HI	(High Alarm Limit)
BAL_MH1_MOLD01_HYD_PRESSURE_HIHI	(High-High Alarm Limit)
BAL_MH1_MOLD01_CYCLE_COUNT_PV	(Cycle Counter Value)
BAL_MH1_MOLD01_STATE_PV	(Equipment State)
BAL_MH1_MOLD01_FAULT_CODE	(Active Fault Code)

3.1.3 Tag Naming Elements

Element	Format	Description
Site	3 characters	BAL=Ballina, SDR=Stuarts Draft, LIB=Libertyville, KAU=Kaunas, FRE=Frederiksberg
Area	2-4 characters	Production area code (MH1=Molding Hall 1, PKG=Packaging, CLN=Cleanroom)
Equipment	6-10 characters	Equipment identifier matching asset registry (MOLD01, PRESS02, ROBOT03)
Module	3-6 characters	Functional module within equipment (HYD=Hydraulic, CLAMP, INJECT, COOL)
Point	Variable	Measurement or control point (PRESSURE, TEMP, FLOW, SPEED, POSITION)

Attribute	2-4 characters	PV=Process Value, SP=Setpoint, HI/LO=Alarm, CMD=Command, STS=Status
------------------	----------------	---

3.1.4 Equipment State Model (PackML/ISA-88)

All equipment must implement a standardized state model based on PackML (ISA-TR88.00.02) for consistent status reporting:

State ID	State Name	Description
0	Undefined	State not determined or communication failure
1	Clearing	Fault is being cleared, machine preparing for restart
2	Stopped	Machine at rest, ready for commands
3	Starting	Machine ramping up to execute state
4	Idle	Machine running but not producing (waiting for material/command)
5	Suspended	Production paused due to upstream/downstream conditions
6	Execute	Machine actively producing
7	Stopping	Production stopping normally
8	Aborting	Emergency stop or fault condition stopping production
9	Aborted	Machine stopped due to abort condition, requires reset
10	Holding	Entering held state (operator pause)
11	Held	Production paused by operator, awaiting resume
12	Completing	Finishing current production cycle
13	Complete	Production batch/order complete
14	Resetting	Returning to idle state

4. Edge Computing Layer (Layer 3)

4.1 Edge Gateway Architecture

The edge computing layer provides the critical abstraction between physical devices and enterprise systems. Edge gateways are deployed at each manufacturing facility to perform protocol translation, data normalization, and local processing.

4.1.1 Edge Gateway Functions

Function	Description
Protocol Translation	Convert proprietary PLC protocols (EtherNet/IP, S7, Modbus) to standard protocols (OPC-UA, MQTT)
Data Normalization	Map vendor-specific data types to canonical formats, apply engineering unit conversions, standardize timestamps
Store and Forward	Buffer data locally when upstream connectivity is lost, replay when connection restored
Edge Analytics	Perform time-critical calculations locally (OEE, SPC, aggregations) to reduce latency
Data Filtering	Apply deadband filtering, downsampling, and exception-based reporting to reduce bandwidth
Security Gateway	Provide DMZ between OT and IT networks, enforce access policies, encrypt data in transit

4.1.2 Recommended Edge Platforms

The following edge platforms are approved for deployment based on protocol support, scalability, and vendor neutrality:

Platform	Strengths	Use Case
Kepware KEPServerEX	150+ device drivers, industry standard, proven reliability	Primary protocol translator for legacy and modern PLCs
Ignition Edge	MQTT-native, unlimited tags, open licensing model	High-volume data collection, store-and-forward scenarios
Azure IoT Edge	Container-based, AI/ML at edge, cloud integration	Edge analytics, predictive maintenance models
EdgeX Foundry	Open source (Linux Foundation), vendor neutral, microservices	Avoiding vendor lock-in, custom integration needs

4.2 Protocol Standards

4.2.1 OPC-UA (Primary Standard)

OPC Unified Architecture is the primary protocol for PLC-to-edge communication. All new equipment must support OPC-UA natively or through gateway translation.

OPC-UA Configuration Requirements:

- Security Mode: SignAndEncrypt (minimum Sign for legacy)
- Security Policy: Basic256Sha256 (preferred) or Basic256

- User Authentication: Username/Password with certificate validation
- Session Timeout: 30 minutes (configurable)
- Publishing Interval: 100ms minimum for process data, 1s for status

4.2.2 MQTT (Event Messaging)

MQTT is used for lightweight, event-driven messaging from edge to enterprise, particularly for high-volume sensor data and real-time events.

MQTT Configuration Requirements:

- Protocol Version: MQTT 5.0 (preferred) or 3.1.1
- QoS Level: QoS 1 (at least once) for production data, QoS 2 for critical events
- TLS: Required for all connections (TLS 1.2 minimum)
- Authentication: Client certificates or username/password
- Topic Structure: hollister/{site}/{area}/{equipment}/{datatype}

MQTT Topic Hierarchy:



5. Semantic and Context Layer (Layer 4)

5.1 Canonical Data Model

The semantic layer provides meaning to raw device data through canonical models based on ISA-95/IEC 62264 standards. This layer is the key to vendor independence—all integrations reference canonical models rather than physical device structures.

5.1.1 Equipment Hierarchy (ISA-95)

All equipment is organized according to the ISA-95 equipment hierarchy:

```
Enterprise (Hollister Incorporated)
├── Site (Ballina, Stuarts Draft, Libertyville, Kaunas, Frederiksberg)
│   ├── Area (Molding Hall 1, Packaging, Cleanroom)
│   │   ├── Work Center (Catheter Line 01, Ostomy Line 02)
│   │   │   ├── Work Unit (MOLD-001, PRESS-002, ROBOT-003)
│   │   │   │   ├── Equipment Module (Hydraulic System, Cooling System)
│   │   │   │   └── Control Module (Pressure Sensor, Temperature Probe)
```

5.1.2 Equipment Definition Schema

Each piece of equipment is defined in the canonical registry using this JSON schema:

```

{
  "$schema": "https://hollister.com/schemas/equipment/v1",
  "equipmentId": "MOLD-001",
  "equipmentClass": "InjectionMoldingMachine",
  "description": "Arburg 520A Injection Molding Machine",
  "hierarchy": {
    "enterprise": "Hollister",
    "site": "BAL",
    "area": "MH1",
    "workCenter": "CATH-LINE-01",
    "workUnit": "MOLD-001"
  },
  "location": {
    "building": "Building A",
    "floor": "Ground",
    "coordinates": { "x": 45.2, "y": 120.8 }
  },
  "capabilities": [
    {
      "capabilityType": "Molding",
      "capabilityValue": {
        "materials": ["HDPE", "LDPE", "PP", "TPE"],
        "maxShotSize": 250,
        "maxClampForce": 520,
        "maxMoldSize": { "width": 500, "height": 400 }
      }
    }
  ],
  "stateMachine": {
    "standard": "PackML",
    "stateTagPath": "opc.tcp://edge-bal-01:4840/ns=2;s=BAL_MH1_MOLD01_STATE_PV"
  },
  "dataPoints": [
    {
      "canonicalName": "CycleTime",
      "description": "Time for complete injection cycle",
      "dataType": "Float32",
      "engineeringUnit": "seconds",
      "physicalTag": "BAL_MH1_MOLD01_CYCLE_TIME_PV",
      "protocol": "OPC-UA",
      "endpoint": "opc.tcp://edge-bal-01:4840",
      "nodeId": "ns=2;s=BAL_MH1_MOLD01_CYCLE_TIME_PV",
      "sampleRate": "1s",
      "deadband": 0.1
    },
    {
      "canonicalName": "MeltTemperature",
      "description": "Barrel melt temperature",
      "dataType": "Float32",
      "engineeringUnit": "degC",
      "physicalTag": "BAL_MH1_MOLD01_BARREL_TEMP_PV",
      "protocol": "OPC-UA",
      "endpoint": "opc.tcp://edge-bal-01:4840",
      "nodeId": "ns=2;s=BAL_MH1_MOLD01_BARREL_TEMP_PV",
      "sampleRate": "5s",
      "limits": { "low": 180, "high": 280, "lowLow": 170, "highHigh": 290 }
    }
  ],
  "maintenanceSchedule": {

```

5.2 Metadata Registry

The metadata registry is the central repository for all data definitions, providing the translation layer between physical tags and canonical names.

5.2.1 Registry Components

Component	Description
Asset Registry	Complete inventory of equipment, sensors, and PLCs with hierarchy
Tag Dictionary	Mapping between physical PLC tags and canonical data point names
Unit of Measure Catalog	Standard engineering units with conversion factors (SI preferred)
Data Type Registry	Canonical data types with validation rules and range limits
Equipment Class Library	Templates for equipment types with standard data points and capabilities
Protocol Endpoint Catalog	Connection details for all OPC-UA servers and MQTT brokers

5.3 Data Governance

The governance engine enforces data quality, security, and compliance policies across all integration flows.

5.3.1 Governance Policies

- **Data Lineage:** Track origin of every data point from physical sensor through transformations to consumers
- **Access Control:** Role-based access to equipment data aligned with organizational structure
- **Data Quality:** Automated validation of values against defined ranges and patterns
- **Change Control:** All metadata changes require approval workflow with audit trail
- **Retention Policies:** Define storage duration based on regulatory requirements (FDA: 2 years beyond device lifetime)

6. Integration Platform (Layer 5)

6.1 Event Streaming Architecture

Apache Kafka serves as the central event streaming platform, providing durable, scalable, and decoupled communication between data producers and consumers.

6.1.1 Kafka Topic Structure

Topic Pattern	Partitions	Purpose
equipment.state.{site}	By equipment ID	Equipment state changes (PackML states)
production.events.{site}	By work order	Cycle complete, quantity produced, material consumed
process.measurements.{site}	By equipment ID	High-frequency process data (temp, pressure, speed)
quality.inspections.{site}	By work order	Inspection results, SPC data, pass/fail
alarms.active.{site}	By severity	Active alarms and acknowledgments
maintenance.events.{site}	By equipment ID	Maintenance requests, completions, PM due
commands.equipment.{site}	By equipment ID	Southbound commands to equipment

6.1.2 Message Schema (Avro)

All messages conform to registered Avro schemas to ensure data contracts between producers and consumers:

```
{
  "type": "record",
  "name": "ProductionEvent",
  "namespace": "com.hollister.manufacturing.events",
  "doc": "Event generated when production cycle completes",
  "fields": [
    {
      "name": "eventId",
      "type": "string",
      "doc": "Unique identifier for this event (UUID)"
    },
    {
      "name": "eventType",
      "type": {
        "type": "enum",
        "name": "ProductionEventType",
        "symbols": [ "CYCLE_COMPLETE", "BATCH_START", "BATCH_END",
                     "MATERIAL_CONSUMED", "SCRAP_PRODUCED" ]
      }
    },
    {
      "name": "timestamp",
      "type": "long",
      "logicalType": "timestamp-millis",
      "doc": "Event timestamp in UTC milliseconds"
    },
    {
      "name": "equipmentId",
      "type": "string",
      "doc": "Canonical equipment identifier"
    },
    {
      "name": "workOrderId",
      "type": [ "null", "string" ],
      "default": null,
      "doc": "Associated work order if applicable"
    },
    {
      "name": "materialLot",
      "type": [ "null", "string" ],
      "default": null,
      "doc": "Material lot number consumed"
    },
    {
      "name": "quantityGood",
      "type": [ "null", "int" ],
      "default": null
    },
    {
      "name": "quantityReject",
      "type": [ "null", "int" ],
      "default": null
    },
    {
      "name": "cycleTimeSeconds",
      "type": [ "null", "float" ],
      "default": null
    },
  ],
}
```

6.2 API Gateway

The API gateway provides synchronous access to manufacturing data for applications that require request-response patterns.

6.2.1 API Endpoints

Method	Endpoint	Description
GET	/api/v1/equipment/{id}	Get equipment definition and current state
GET	/api/v1/equipment/{id}/datapoints	Get current values of all data points
GET	/api/v1/equipment/{id}/history	Get historical data with time range
POST	/api/v1/equipment/{id}/commands	Send command to equipment (setpoint, recipe)
GET	/api/v1/production/orders/{id}	Get production order status and progress
GET	/api/v1/quality/inspections	Query inspection results with filters
GET	/api/v1/alarms/active	Get all active alarms across equipment

7. MES Integration (Layer 6)

7.1 MES Adapter Pattern

The MES adapter translates between the canonical data model and MES-specific APIs. This pattern ensures that MES changes or replacements do not impact the underlying equipment integration.

7.1.1 Adapter Responsibilities

- Subscribe to relevant Kafka topics for equipment events
- Transform canonical events to MES-specific API calls
- Handle MES-specific authentication and session management
- Manage retry logic and error handling for MES communication
- Transform MES commands to canonical format for southbound flow

7.1.2 Critical Manufacturing MES Integration Points

Integration	Direction	Data and Mapping
Equipment State	PLC → MES	PackML state → CM Resource State (Available, Busy, Down, Maintenance)
Production Tracking	PLC → MES	Cycle complete event → CM TrackIn/TrackOut with quantity, lot, process data
Material Consumption	PLC → MES	Material consumed event → CM Material Tracking with lot genealogy
Quality Data	PLC → MES	Inspection result → CM Data Collection with parametric values
Work Order Download	MES → PLC	CM Work Order Release → Recipe parameters, setpoints, quantities
Label Data	MES → PLC	CM Lot/Serial generation → UDI, barcode data for label

7.1.3 Adapter Implementation Pattern

The following pseudocode illustrates the adapter pattern for production event handling:


```
class MESEquipmentAdapter:
    """
    Adapter translating between canonical events and Critical Manufacturing MES API.
    This is the ONLY component that knows MES-specific details.
    """

    def __init__(self, registry: MetadataRegistry, mes_client: CMClient):
        self.registry = registry
        self.mes = mes_client
        self.kafka_consumer = KafkaConsumer(['production.events.*'])

    async def run(self):
        """Main event processing loop"""
        async for message in self.kafka_consumer:
            event = ProductionEvent.deserialize(message.value)
            await self.handle_production_event(event)

    async def handle_production_event(self, event: ProductionEvent):
        # 1. Lookup equipment in canonical registry
        equipment = self.registry.get_equipment(event.equipment_id)

        # 2. Map canonical equipment to MES resource
        mes_resource = self.get_mes_resource_mapping(equipment)

        # 3. Transform and send to MES based on event type
        if event.event_type == ProductionEventType.CYCLE_COMPLETE:
            await self.report_production_complete(event, equipment, mes_resource)
        elif event.event_type == ProductionEventType.MATERIAL_CONSUMED:
            await self.report_material_consumption(event, equipment, mes_resource)

    async def report_production_complete(self, event, equipment, mes_resource):
        # Transform canonical process data to MES parameters
        mes_data_collection = {}
        for canonical_name, value in event.process_data.items():
            # Lookup MES parameter name from canonical mapping
            data_point = equipment.get_data_point(canonical_name)
            mes_param = self.registry.get_mes_parameter_mapping(
                data_point.canonical_name,
                mes_resource.resource_type
            )
            if mes_param:
                # Apply any unit conversion if needed
                converted_value = self.convert_units(
                    value,
                    data_point.engineering_unit,
                    mes_param.expected_unit
                )
                mes_data_collection[mes_param.name] = converted_value

        # Call Critical Manufacturing API
        await self.mes.track_out(
            resource_id=mes_resource.id,
            work_order_id=event.work_order_id,
            quantity_good=event.quantity_good,
            quantity_reject=event.quantity_reject,
            cycle_time=event.cycle_time_seconds,
            data_collection=mes_data_collection,
            material_lot=event.material_lot
        )
```


8. Security Architecture

8.1 OT/IT Network Segmentation

The architecture implements defense-in-depth with clear separation between Operational Technology (OT) and Information Technology (IT) networks.

8.1.1 Network Zones

Zone	Components	Security Controls
Level 0-1	PLCs, Sensors, Actuators	Air-gapped from IT, physical security, no direct internet
Level 2	SCADA, HMI, Historians	OT network, firewall protected, limited IT access
Level 3 (DMZ)	Edge Gateways, Protocol Translators	Industrial DMZ, dual firewalls, data diode where required
Level 4	MES, Integration Platform	IT network with OT access policies, encrypted communication
Level 5	ERP, Analytics, Cloud	Enterprise IT security controls, internet access

8.2 Authentication and Authorization

- OPC-UA: X.509 certificates for server/client authentication, username/password for user auth
- MQTT: TLS client certificates, OAuth 2.0 tokens for service accounts
- Kafka: SASL/SCRAM authentication, ACLs for topic-level authorization
- APIs: OAuth 2.0 / OpenID Connect, JWT tokens with short expiration
- MES: Integration with enterprise identity provider (Azure AD/Okta)

8.3 Regulatory Compliance (21 CFR Part 11)

All components handling electronic records must comply with FDA 21 CFR Part 11:

- Audit Trails: Every data transaction logged with timestamp, user, before/after values
- Electronic Signatures: Biometric or two-factor authentication for approvals
- Access Controls: Role-based permissions aligned with job responsibilities
- Record Integrity: Cryptographic hashing to detect unauthorized modifications
- System Validation: IQ/OQ/PQ protocols for all GxP-critical systems

9. Implementation Guide

9.1 Equipment Onboarding Checklist

Follow this checklist when integrating new equipment into the Digital Factory:

Step	Task	Deliverable
1	Document equipment capabilities and I/O points	Equipment specification sheet
2	Assign equipment ID and hierarchy location	Asset registry entry
3	Define tag naming following conventions	Tag list document
4	Configure PLC with standard state model	PLC program with PackML states
5	Configure edge gateway connection	OPC-UA/MQTT connectivity verified
6	Create canonical equipment definition	JSON definition in metadata registry
7	Map data points to MES parameters	Mapping configuration
8	Configure Kafka topics and subscriptions	Topic configuration, ACLs
9	Test end-to-end data flow	Integration test results
10	Execute validation protocol (if GxP)	IQ/OQ/PQ documentation
11	Go-live and monitor	Operational handover

9.2 Troubleshooting Guide

Symptom	Possible Cause	Resolution
No data from PLC	OPC-UA connection failure	Check certificates, firewall rules, PLC OPC-UA server status
Data appears in Kafka but not MES	MES adapter not processing	Check adapter logs, verify schema compatibility, MES API connectivity
Incorrect values in MES	Unit conversion or mapping error	Verify canonical model, check unit conversion configuration
High latency in data	Network congestion or polling interval	Review edge gateway sampling rates, network bandwidth
Missing events after network outage	Store-and-forward not enabled	Configure edge gateway buffering, verify replay on reconnect

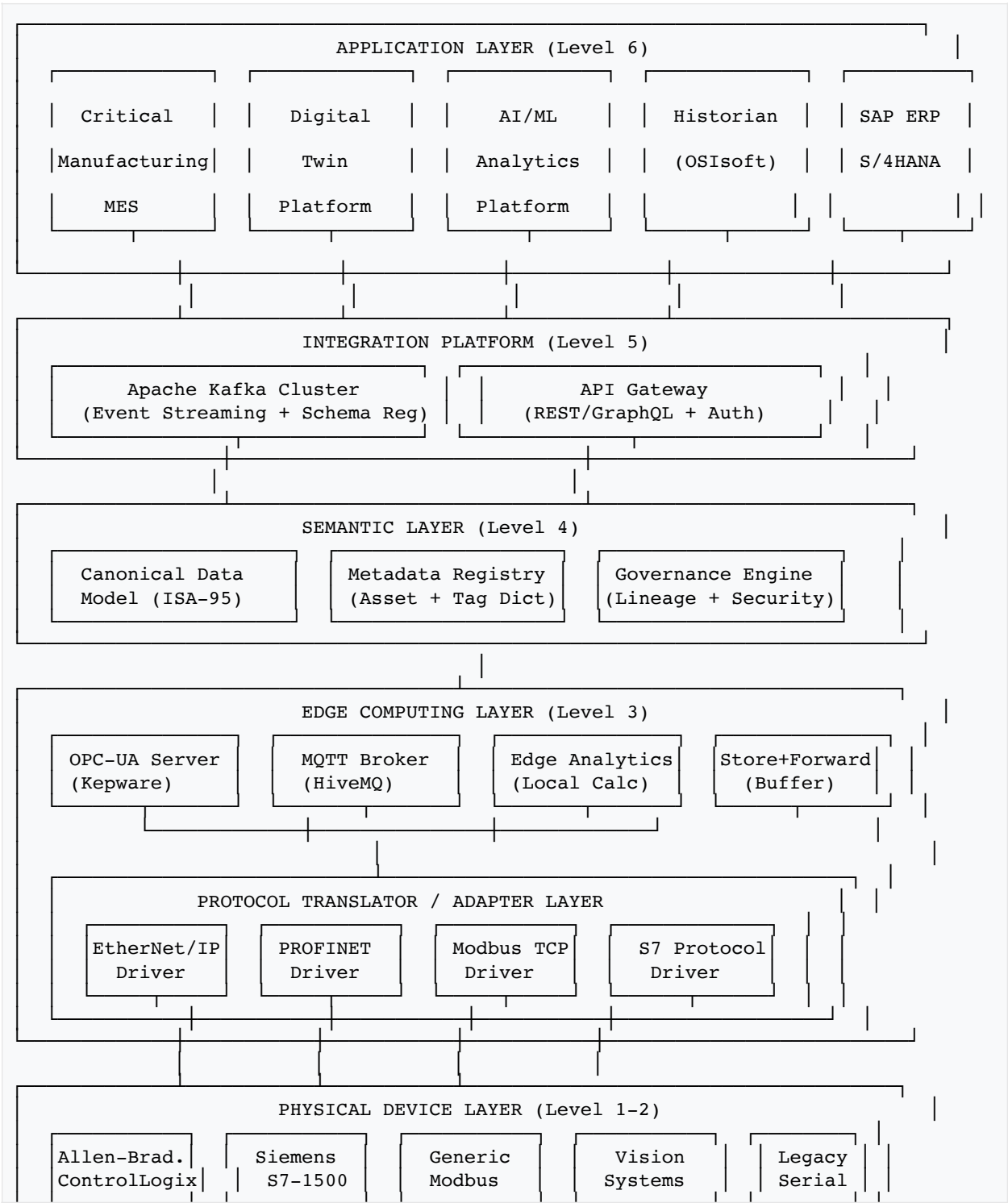
Appendix A: OPC-UA Information Models

The following OPC-UA companion specifications should be implemented where applicable:

Specification	OPC Number	Application
OPC-UA for Machinery	OPC 40001-1	Base model for all production machinery
OPC-UA for PackML	OPC 40200	Packaging machinery state model
OPC-UA for ISA-95	OPC 40600	MES integration data model
OPC-UA for Robotics	OPC 40010	Industrial robot integration
OPC-UA for Machine Vision	OPC 40100	Vision inspection systems

Appendix B: Reference Architecture Diagram

The following diagram illustrates the complete PLC-to-MES integration architecture:



End of Technical Specification